

CLAUDE.md

This file provides guidance to Claude Code when working with the SweetSync repository.

Project overview

SweetSync is a mobile-first group scheduling app built with Expo and React Native. It uses AI to find overlapping free time across multiple people's schedules, then helps the group decide what to do with that time. The core loop is: create a room → members upload schedules → AI finds free slots → group votes on a time and activity → event confirmed. The backend runs entirely on Supabase — no separate server.

Tech stack

Technology	Purpose
Expo + React Native	Mobile framework, single codebase for iOS and Android
TypeScript	Type safety across the entire codebase
Supabase (PostgreSQL)	Database, auth, real-time subscriptions, storage, edge functions
Supabase Auth	Google and Apple OAuth, JWT session management
Supabase Storage	Timetable image and PDF uploads before Gemini processing
Supabase Edge Functions	Server-side Gemini calls, notification triggers, rate limiting
Gemini 2.5 Flash	OCR schedule extraction, slot finding, activity suggestions
Expo Notifications	Push notification delivery (FCM + APNs)
pg_cron	Scheduled notification triggers (nudges, reminders)
Expo AuthSession	OAuth redirect handling and deep link preservation
Fraunces (Google Fonts)	Display font — brand and celebration moments only
Plus Jakarta Sans	Body font — all functional UI text
Phosphor Icons	Icon system (<code>phosphor-react-native</code>)

Technology	Purpose
React Native Reanimated	Animations and transitions

Commands

bash

```
# Development
npx expo start

# Development on iOS simulator
npx expo start --ios

# Development on Android emulator
npx expo start --android

# Build for production (EAS)
eas build --platform ios
eas build --platform android

# Submit to stores
eas submit --platform ios
eas submit --platform android

# Type check
npx tsc --noEmit

# Lint
npx eslint . --ext .ts,.tsx

# Format
npx prettier --write .

# Run Supabase locally
supabase start

# Deploy edge functions
supabase functions deploy

# Generate Supabase types
supabase gen types typescript --local > types/database.ts
```

Project structure

```
sweetsync/
├─ app/                # Expo Router screens (file-based routing)
│   └─ _layout.tsx     # Root layout – font loading, auth provider
│   └─ index.tsx       # Splash / redirect logic
```

- | |— (auth)/ # Unauthenticated screens
 - | | |— onboarding.tsx # Onboarding hero screen
 - | | |— sign-in.tsx # Google / Apple sign-in
- | |— (tabs)/ # Authenticated tab screens
 - | | |— _layout.tsx # Tab bar configuration
 - | | |— index.tsx # Home – rooms list
 - | | |— calendar.tsx # Global calendar tab
 - | | |— votes.tsx # All open voting sessions
 - | | |— profile.tsx # Account and settings
- | |— room/
 - | | |— [id].tsx # Room interior
 - | | |— [id]/calendar.tsx # Group heat map + my schedule
 - | | |— [id]/results.tsx # AI results screen
 - | | |— [id]/vote-slots.tsx # Time slot voting
 - | | |— [id]/vote-activity.tsx # Activity voting
- | |— confirmed/[id].tsx # Confirmed event celebration screen
- | |— join.tsx # Invite link deep link handler
- |— components/ # Reusable UI components
 - | |— Card.tsx # Peach / indigo / mint / neutral card
 - | |— Avatar.tsx # Member avatar circle
 - | |— AvatarStack.tsx # Overlapping avatar row
 - | |— ProgressBar.tsx # Schedule upload progress
 - | |— HeatMap.tsx # Group availability heat map grid
 - | |— SlotCard.tsx # AI-found time slot card
 - | |— ActivityCard.tsx # Activity suggestion card
 - | |— VotePills.tsx # Free / Prefer / Can't response pills
 - | |— StatusPill.tsx # Room status badge
 - | |— Button.tsx # Peach / indigo / mint / outline buttons
- |— constants/
 - | |— theme.ts # Colors, fonts, spacing, radius tokens
 - | |— animation.ts # Transition durations and easing curves
- |— hooks/
 - | |— useAuth.ts # Supabase auth state
 - | |— useRoom.ts # Room data + real-time subscription
 - | |— useHeatMap.ts # Heat map computation from schedule blocks
 - | |— useVoting.ts # Voting state and submission
 - | |— useNotifications.ts # Push token registration
- |— lib/
 - | |— supabase.ts # Supabase client initialization
 - | |— gemini.ts # Gemini API call wrappers (client-safe)
 - | |— heatmap.ts # getHeatShade() and grid computation
- |— supabase/
 - | |— functions/ # Edge Functions (Deno runtime)
 - | | |— extract-schedule/ # OCR extraction via Gemini

```

|   |   |— find-slots/           # AI slot finding and ranking
|   |   |— suggest-activities/  # AI activity suggestions
|   |   |— send-notification/   # Expo Push API wrapper
|   |   |— fire-notifications/  # pg_cron target – fires pending notifications
|   |   |— migrations/         # Supabase SQL migrations
|— types/
|   |— database.ts             # Generated Supabase types (do not edit manually)
|   |— index.ts                # App-level types (Room, Member, Slot, Vote, etc.)
|— assets/
|   |— fonts/                  # Font files if self-hosting (otherwise Expo
Google Fonts)
|— DESIGN.md                   # Design system reference
|— TECH.md                     # Technology decisions and scaling notes
|— sweetsync-prd.md            # Product requirements document
|— CLAUDE.md                   # This file

```

Architecture

SweetSync uses a feature-based screen architecture with Expo Router (file-based routing). The data layer is entirely Supabase — the app never calls a custom backend server. All sensitive operations (Gemini API calls, notification dispatch, rate limiting) run inside Supabase Edge Functions so API keys never reach the client.

Data flow:

```

Screen → Supabase client (anon key) → RLS-enforced PostgreSQL
                                         → Real-time channel subscription
                                         → Storage (file upload)
                                         → Edge Function trigger
                                           → Gemini API (server-side only)
                                           → Expo Push API

```

Real-time pattern: Supabase channels power three live features — the schedule upload progress indicator, the group heat map updating as members confirm, and the live vote tally during voting. Always unsubscribe channels on component unmount.

Heat map computation: Starts client-side in `lib/heatmap.ts`. The `getHeatShade(freeCount, totalMembers)` function is isolated so it can move to an Edge Function later without schema changes. Results will eventually be cached in `heat_map_cache`.

Session state machine: Room sessions move through six states — `collecting → processing → voting_slots → voting_activity → confirmed → expired`. State transitions have guard

conditions enforced at the database level via RLS policies and in Edge Functions. Never transition state client-side only.

Code patterns

Naming conventions

- Files and folders: `kebab-case` for routes, `PascalCase` for components
- Variables and functions: `camelCase`
- Types and interfaces: `PascalCase`
- Constants: `SCREAMING_SNAKE_CASE` for true constants, `camelCase` for theme tokens
- Database columns: `snake_case` (Supabase convention)
- Edge Function folders: `kebab-case` matching the function name

Component structure

```
typescript

// Always import from constants – never hardcode colors or spacing
import { colors, fonts, spacing, radius } from '@constants/theme';

// Card variant pattern – use the Card component, never inline card styles
import { Card } from '@components/Card';
<Card variant="indigo">...</Card> // peach | indigo | mint | neutral

// Font pattern – always use named font constants
fontFamily: fonts.display // Fraunces 700 – brand moments only
fontFamily: fonts.bodySemibold // Plus Jakarta Sans 600 – functional headings
fontFamily: fonts.body // Plus Jakarta Sans 400 – body text
```

Typography rules

Fraunces (`fonts.display` / `fonts.displayItalic`) is used **only** at these moments:

- Splash screen app name
- Onboarding hero headline
- Room names (home screen list and room interior header)
- AI results headline ("We found 3 free slots")
- Activity name on confirmed event screen

- Confirmed event name (*italic*)

Everything else — labels, buttons, captions, body copy, inputs, tab bar — uses Plus Jakarta Sans. If you are unsure, use Plus Jakarta Sans.

Color rules

Color encodes meaning in SweetSync. Do not use colors outside their semantic role:

Color family	Token prefix	Meaning	Where
Peach	colors.peach*	Rooms and people	Room cards, primary CTAs, active tab
Indigo	colors.indigo*	AI and scheduling	Slot cards, AI buttons, processing state
Mint	colors.mint*	Confirmed / success	Event cards, success states, confirmed screen
Neutral gray	◯ #F7F6F3	Passive / waiting	Unuploaded states, neutral cards

Supabase query pattern

```
typescript
// Always handle errors explicitly - never assume success
const { data, error } = await supabase
  .from('schedules')
  .select('*')
  .eq('room_id', roomId);

if (error) {
  console.error('Schedule fetch failed:', error.message);
  return;
}
```

Real-time subscription pattern

```
typescript
```

```
// Always clean up on unmount – memory leak if you don't
useEffect(() => {
  const channel = supabase
    .channel(`room-${roomId}`)
    .on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'schedules',
    .subscribe();

  return () => { supabase.removeChannel(channel); };
}, [roomId]);
```

Edge Function pattern

typescript

```
// All Edge Functions run on Deno – not Node.js
// Import from npm: via npm: specifier or esm.sh
import { createClient } from 'npm:@supabase/supabase-js@2';

// Always validate before calling Gemini
// Always rate-limit before calling Gemini
// Never return raw Gemini errors to the client
export default async function handler(req: Request): Promise<Response> {
  const { roomId, userId } = await req.json();

  const withinLimit = await checkRateLimit(userId, 'schedule-upload');
  if (!withinLimit) {
    return new Response(JSON.stringify({ error: 'Rate limit exceeded' }), { status:
  }

  // ... Gemini call with retry
  return new Response(JSON.stringify({ blocks }), { status: 200 });
}
```

Error handling

- Never surface raw Supabase or Gemini error messages to the user
- All errors shown to users are plain language: "Something went wrong — try again"
- Log full errors to console in development; send to logging service in production
- Gemini 429 errors must retry with exponential backoff — never fail immediately
- File upload failures should allow retry without losing other room state

Animation pattern

typescript

```
import { transitions } from '@/constants/animation';
import Animated, { useSharedValue, withTiming, Easing } from 'react-native-reanimated';

// Card entrance - always stagger list items by 50ms
const opacity = useSharedValue(0);
const translateY = useSharedValue(8);

useEffect(() => {
  opacity.value = withTiming(1, { duration: transitions.cardEntrance.duration });
  translateY.value = withTiming(0, { duration: transitions.cardEntrance.duration });
}, []);
```

Testing

- **Type check:** `npx tsc --noEmit`
- **Lint:** `npx eslint . --ext .ts,.tsx`
- **Test location:** `__tests__/` mirroring the source structure
- **Pattern:** Unit test pure functions (especially `getHeatShade`), vote tallying logic, state transition guards). Integration tests for Supabase queries using a local Supabase instance.

bash

```
# Run type check and lint before any commit
npx tsc --noEmit && npx eslint . --ext .ts,.tsx
```

Critical functions to unit test:

- `lib/heatmap.ts` — `getHeatShade(freeCount, totalMembers)`
- Vote tally calculation — most votes wins, tie detection
- Session state transition guard conditions
- Invite token expiry validation

Validation

Run before every commit:

```
bash

npx tsc --noEmit
npx eslint . --ext .ts,.tsx
npx prettier --check .
```

Run before every pull request:

```
bash

npx tsc --noEmit
npx eslint . --ext .ts,.tsx
supabase db lint
```

Key files

File	Purpose
app/_layout.tsx	Root layout — font loading, auth state, deep link handling
app/join.tsx	Invite link handler — preserves room context through OAuth
constants/theme.ts	Single source of truth for all colors, fonts, spacing, radius
constants/animation.ts	All transition durations and easing curves
lib/supabase.ts	Supabase client — import this, never create a new client
lib/heatmap.ts	getHeatShade() — isolated for future server-side migration
types/database.ts	Generated Supabase types — do not edit manually, regenerate with CLI
supabase/functions/extract-schedule/	Gemini OCR call — rate limited, file validated, retried

File	Purpose
<code>supabase/functions/fire-notifications/</code>	pg_cron target — checks <code>notifications_log</code> and fires pending
<code>DESIGN.md</code>	Full design system — colors, typography, spacing, card styles, animation
<code>TECH.md</code>	Technology decisions, pricing, scalability roadmap
<code>sweetsync-prd.md</code>	Product requirements — feature specs, user journey, navigation

On-demand context

Topic	File
Full design system	<code>DESIGN.md</code>
Tech stack decisions and pricing	<code>TECH.md</code>
Feature requirements and user journey	<code>sweetsync-prd.md</code>
Database schema	<code>TECH.md</code> → Database → Schema
Color tokens	<code>constants/theme.ts</code> and <code>DESIGN.md</code> → Color system
Notification schedule	<code>sweetsync-prd.md</code> → Notification system
Session state machine	<code>TECH.md</code> → Architecture
Heat map design spec	<code>DESIGN.md</code> → Group calendar
Scaling roadmap	<code>TECH.md</code> → Scaling roadmap
Google Stitch screen prompts	<code>sweetsync-stitch-prompts.md</code>

Notes

- **Deno runtime in Edge Functions** — Edge Functions run on Deno, not Node.js. Use `npm:` specifier for npm packages. Check Deno compatibility before adding any package.

- **Never call Gemini from the client** — all Gemini calls go through Edge Functions. The API key must never appear in client-side code.
- **RLS on every table** — every new table must have Row Level Security enabled and policies defined before any data is written to it.
- **Deep link through auth is critical** — the invite link flow must preserve `roomId` through the OAuth redirect. Test this on both iOS and Android before any release.
- **Heat map is client-side for now** — `getHeatShade()` in `lib/heatmap.ts` runs client-side. Keep it isolated. It will move to an Edge Function when scale demands it — do not embed heat map logic directly in components.
- **Session state transitions** — never transition session state (`collecting`, `processing`, `voting_slots`, etc.) client-side only. All state changes go through Supabase and are validated server-side.
- **Font loading blocks render** — `SplashScreen.preventAutoHideAsync()` is called in `_layout.tsx`. The splash stays visible until both Fraunces and Plus Jakarta Sans are loaded. Do not hide the splash screen early.
- **Push permission timing** — request push notification permission after the user's first schedule upload confirms, not during onboarding. This is intentional.
- **No hardcoded colors** — always import from `constants/theme.ts`. No hex values in component files.
- **Regenerate types after schema changes** — run `supabase gen types typescript --local > types/database.ts` after any migration.